Pontificia Universidad Católica de Chile
IA Lab

IA Lab

DL Models for Audio Processing

Gabriel Sepúlveda

Computer Science Department (DCC), PUC

DL Models for Audio

So far, we have discussed:

1. The nature of sound. In particular,
   - Its composition as a superposition of frequency components
   - Popular hand-crafted features used to process audio. Ex. spectrograms and log-mel.
2. Tools to support audio applications:
   - Structure of audio files
   - Software libraries to process audio
   - Datasets.

This class: DL Models to process audio

Applications

Audio Main Applications

According to the nature of audio signals, we can divide their applications into 3 main categories:

- General sounds.
  Ex. environmental sound classification, audio tagging, ...
- Speech.
  Ex. Speech recognition, speech translation, speaker identification, ...
- Music.
  Ex. song recognition, musical instrument identification, ...

This class mostly focuses on environmental sounds. Next class we will discuss speech and voice. We will discuss music just briefly.

General Sounds

Audio Applications: General Sounds

Sound enhancement:

- Noise reduction/elimination.
- Sound reconstruction.
- Source separation.
- Source transformation.
- ...

Audio Applications: General Sounds

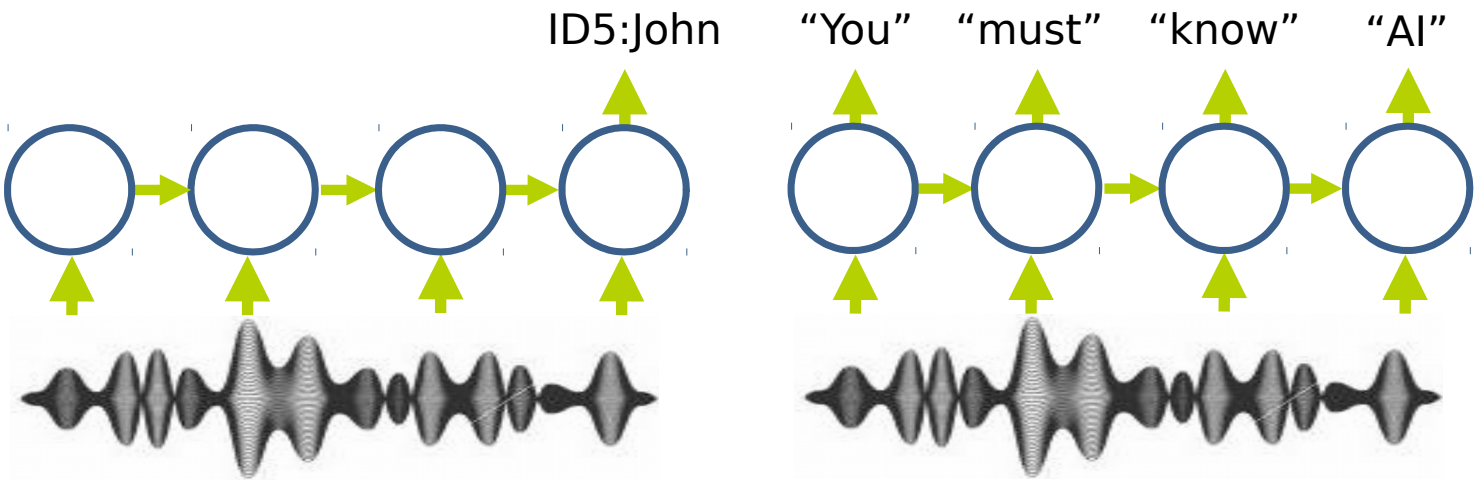
Synthesis:

- Sound synthesis.
- Speech synthesis.
- Music creation.
- ...

Audio Applications: General Sounds

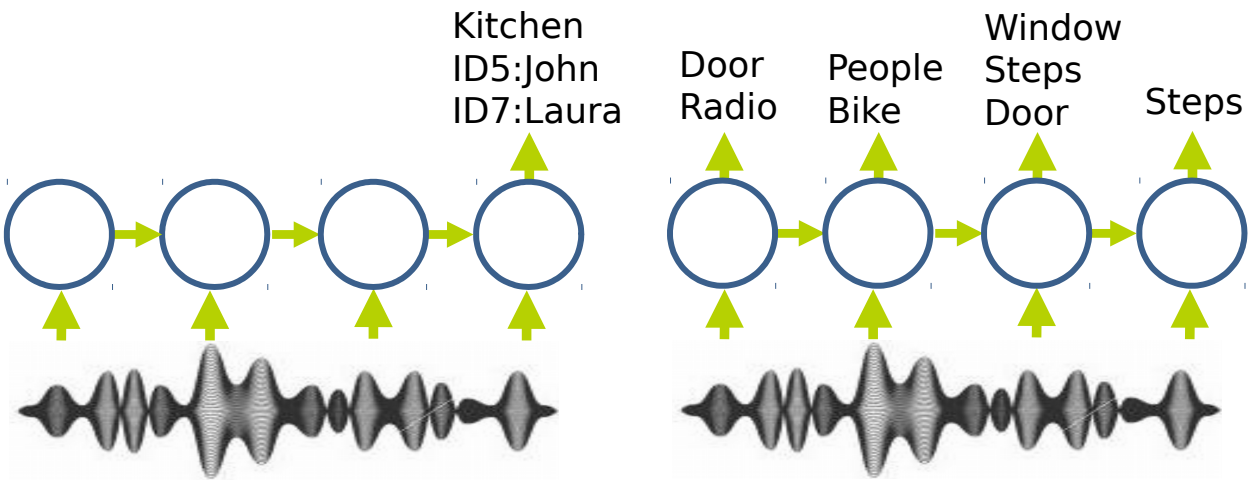
Classification:

- Single global-label vs single local-labels:
  - Ex.: John is talking vs John say: "you must know AI".



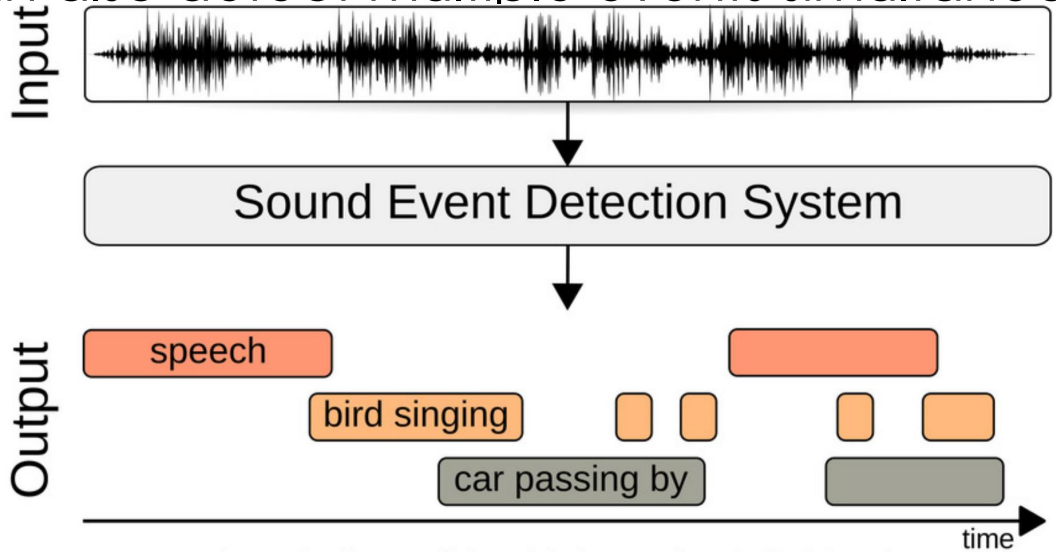
Audio Applications: General Sounds

- Multiple global-labels vs multiple local-labels:
  - Temporal tagging of audio sequences.
  - Multiple speaker identification.
  - Sound recognition at different levels of granularity. Ex. Car vs specific part of a car.



Audio Applications: General Sounds

- Event detection: detect a specific sound in a long audio sequence.
  - Somebody is opening a door.
  - There is a heartbeat anomaly.
  - We can also detect multiple events simultaneously.



Example: http://soundnet.csail.mit.edu/

Speech

Audio Applications: Speech

- Speech recognition.
- Video captioning.
- Speaker audio separation.
- Language translation.
- ...

Music

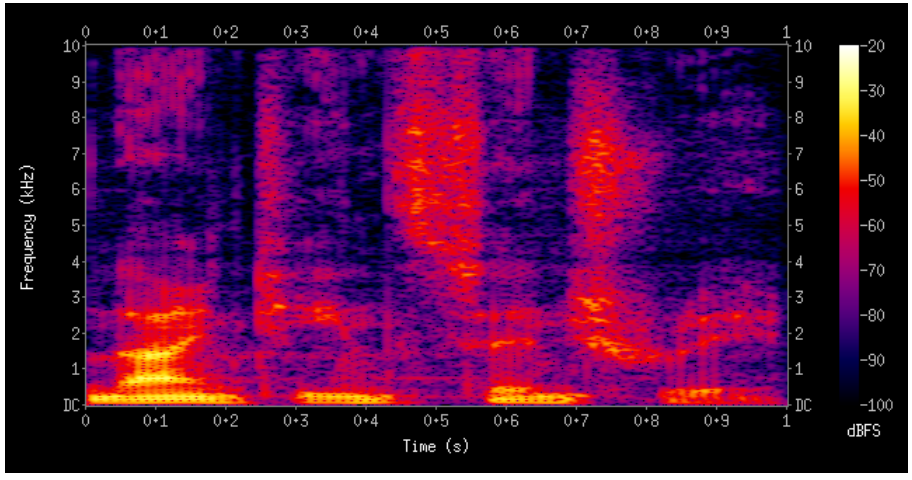
Audio Applications: Music

- Song recognition.
- Song/music-style similarity.
- Music instrument detection.
- Music transcription.
- ...

Models

Audio vs Image Data

- We have already analized models for visual recognition.
- In principle, the 2D time-freq representation (spectrogram) of an audio signal can be interpreted as an image:



- In this way, we can use 2D CNNs to process audio signals.
- While this is possible, there are relevant differences between audio and visual data that is important to consider.

Modeling Options for Audio Signals

So far, we have discussed several modeling options, which ones?

Modeling Options for Audio Signals

So far, we have discussed several modeling options, which ones?

- MLPs (Feedforward fully connected NNs)

Modeling Options for Audio Signals

So far, we have discussed several modeling options, which ones?

- MLPs (Feedforward fully connected NNs)
- CNNs

Modeling Options for Audio Signals

So far, we have discussed several modeling options, which ones?

- MLPs (Feedforward fully connected NNs)
- CNNs
- RNNs

Modeling Options for Audio Signals

So far, we have discussed several modeling options, which ones?

- MLPs (Feedforward fully connected NNs)
- CNNs
- RNNs
- GANs

Modeling Options for Audio Signals

So far, we have discussed several modeling options, which ones?

- MLPs (Feedforward fully connected NNs)
- CNNs
- RNNs
- GANs
- Transformer and relational models

Modeling Options for Audio Signals

So far, we have discussed several modeling options, which ones?

- MLPs (Feedforward fully connected NNs)
- CNNs
- RNNs
- GANs
- Transformer and relational models
- Reinforcement learning

Modeling Options for Audio Signals

So far, we have discussed several modeling options, which ones?

- MLPs (Feedforward fully connected NNs)
- CNNs
- RNNs
- GANs
- Transformer and relational models
- Reinforcement learning
- Imitation learning

Modeling Options for Audio Signals

So far, we have discussed several modeling options, which ones?

- MLPs (Feedforward fully connected NNs)
- CNNs
- RNNs
- GANs
- Transformer and relational models
- Reinforcement learning
- Imitation learning
- Reasoning: Neuro-Symbolic models

Modeling Options for Audio Signals

For audio applications, most popular models are based on a combination of:

Modeling Options for Audio Signals

For audio applications, most popular models are based on a combination of:

- MLPs

Modeling Options for Audio Signals

For audio applications, most popular models are based on a combination of:

- MLPs
- CNNs

Modeling Options for Audio Signals

For audio applications, most popular models are based on a combination of:

- MLPs
- CNNs
- RNNs

Modeling Options for Audio Signals

For audio applications, most popular models are based on a combination of:

- MLPs
- CNNs
- RNNs

Modeling Options for Audio Signals

For audio applications, most popular models are based on a combination of:

- MLPs
- CNNs
- RNNs

Why?

They have complementary properties to analyze audio signals.

Let's check more on this.

- CNNs: good properties to learn local features.

  Specifically, by exploiting: i) Local span of each filter + ii) Translation invariance of convolutional operator $\rightarrow$ they discover relevant local features in the input data.

- CNNs: good properties to learn local features.

  Specifically, by exploiting: i) Local span of each filter + ii) Translation invariance of convolutional operator $\rightarrow$ they discover relevant local features in the input data.

- RNNs: good properties to learn temporal features.

  Specifically, by exploiting a recurrent operator, they discover relevant distant temporal relations (global) in the input data.

- CNNs: good properties to learn local features.

  Specifically, by exploiting: i) Local span of each filter + ii) Translation invariance of convolutional operator $\rightarrow$ they discover relevant local features in the input data.

- RNNs: good properties to learn temporal features.

  Specifically, by exploiting a recurrent operator, they discover relevant distant temporal relations (global) in the input data.

- MLPs: good properties to classify input data.

  Specifically, by exploiting informative features, they learn good classifiers that map input features to discriminative spaces (class discrimination).

- CNNs: good properties to learn local features.

  Specifically, by exploiting: i) Local span of each filter + ii) Translation invariance of convolutional operator $\rightarrow$ they discover relevant local features in the input data.

- RNNs: good properties to learn temporal features.

  Specifically, by exploiting a recurrent operator, they discover relevant distant temporal relations (global) in the input data.

- MLPs: good properties to classify input data.

  Specifically, by exploiting informative features, they learn good classifiers that map input features to discriminative spaces (class discrimination).

- CNNs: good properties to learn local features.

  Specifically, by exploiting: i) Local span of each filter + ii) Translation invariance of convolutional operator $\rightarrow$ they discover relevant local features in the input data.

- RNNs: good properties to learn temporal features.

  Specifically, by exploiting a recurrent operator, they discover relevant distant temporal relations (global) in the input data.

- MLPs: good properties to classify input data.

  Specifically, by exploiting informative features, they learn good classifiers that map input features to discriminative spaces (class discrimination).

In summary:

- CNNs: Learn meaningful local features.

- CNNs: good properties to learn local features.

  Specifically, by exploiting: i) Local span of each filter + ii) Translation invariance of convolutional operator $\rightarrow$ they discover relevant local features in the input data.

- RNNs: good properties to learn temporal features.

  Specifically, by exploiting a recurrent operator, they discover relevant distant temporal relations (global) in the input data.

- MLPs: good properties to classify input data.

  Specifically, by exploiting informative features, they learn good classifiers that map input features to discriminative spaces (class discrimination).

In summary:

- CNNs: Learn meaningful local features.
- RNNs: Learn distance and global temporal features.

- CNNs: good properties to learn local features.

  Specifically, by exploiting: i) Local span of each filter + ii) Translation invariance of convolutional operator $\rightarrow$ they discover relevant local features in the input data.

- RNNs: good properties to learn temporal features.

  Specifically, by exploiting a recurrent operator, they discover relevant distant temporal relations (global) in the input data.

- MLPs: good properties to classify input data.

  Specifically, by exploiting informative features, they learn good classifiers that map input features to discriminative spaces (class discrimination).

In summary:

- CNNs: Learn meaningful local features.
- RNNs: Learn distance and global temporal features.
- MLPs: Learn good classifiers.

Example 1: DL Arquiteture to Process an Audio Signal

Example 1: DL Arquiteture to Process an Audio Signal

Input:

- 40D Log-mel feats for overlapped segments of 10-20ms. 5-10ms overlap.

Example 1: DL Arquiteture to Process an Audio Signal

Input:

- 40D Log-mel feats for overlapped segments of 10-20ms. 5-10ms overlap.

CNN :

- 2 convolutional layers. Each with: i) 256 filter, ii) 9x9 and 4x4 filter sizes, respectively.
- Optional max-pooling in frequency only. Ex. Non-ovelapped windows of size 3.
- Batch normalization is optional.
- Add 1x1 convolution to reduce dimension. This allows for a reduction in parameters with no loss in accuracy.

Example 1: DL Arquiteture to Process an Audio Signal

Input:

- 40D Log-mel feats for overlapped segments of 10-20ms. 5-10ms overlap.

CNN :

- 2 convolutional layers. Each with: i) 256 filter, ii) 9x9 and 4x4 filter sizes, respectively.
- Optional max-pooling in frequency only. Ex. Non-ovelapped windows of size 3.
- Batch normalization is optional.
- Add 1x1 convolution to reduce dimension. This allows for a reduction in parameters with no loss in accuracy.

RNN :

- 2 LSTM layers. Cells in LSTMs with 256D.
- Need to normalize sequence length in minibatch.

Example 1: DL Arquiteture to Process an Audio Signal

Input:

- 40D Log-mel feats for overlapped segments of 10-20ms. 5-10ms overlap.

CNN :

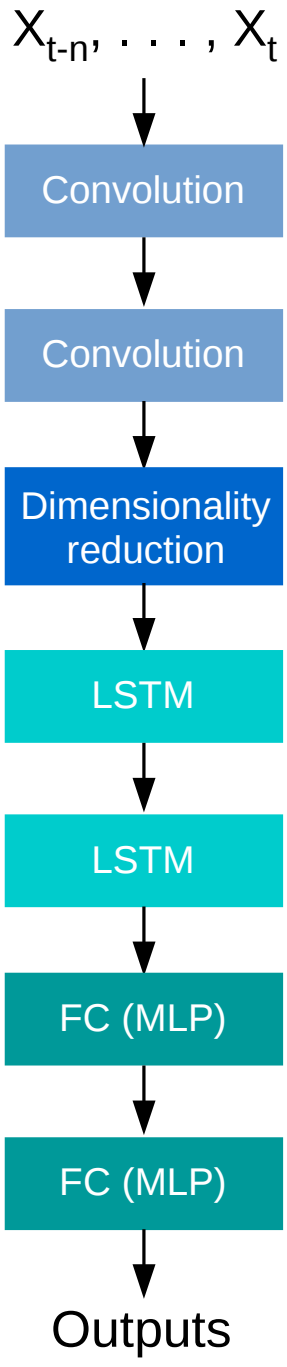
- 2 convolutional layers. Each with: i) 256 filter, ii) 9x9 and 4x4 filter sizes, respectively.
- Optional max-pooling in frequency only. Ex. Non-ovelapped windows of size 3.
- Batch normalization is optional.
- Add 1x1 convolution to reduce dimension. This allows for a reduction in parameters with no loss in accuracy.

RNN :

- 2 LSTM layers. Cells in LSTMs with 256D.
- Need to normalize sequence length in minibatch.

MLP :

- 2 FC layers. Each FC layer has 1.024 hidden units.
- Output: softmax (or sigmoids) for class label(s).



- Details of the structure are usually chosen experimentally based on validation error.
- Sample frequency and application determine window for temporal context.
- Use fewer parameters for less data to avoid overfitting.
- Decrease filter size and increase number of channels for deeper layers.

Can We Use Raw Audio Data

Can We Use Raw Audio Data

- Spectrograms, log-mel, etc. are hand-crafted features.

Can We Use Raw Audio Data

- Spectrograms, log-mel, etc. are hand-crafted features.
- Can we use DL to directly learn features from raw data?

Can We Use Raw Audio Data

- Spectrograms, log-mel, etc. are hand-crafted features.
- Can we use DL to directly learn features from raw data?
- Yes, but we need to consider some issues.

Can We Use Raw Audio Data

- Spectrograms, log-mel, etc. are hand-crafted features.
- Can we use DL to directly learn features from raw data?
- Yes, but we need to consider some issues.
  - To avoid loss of info, we need to sample audio data using a high sample rate 15-20KHz (44.1 Khz for music).

Can We Use Raw Audio Data

- Spectrograms, log-mel, etc. are hand-crafted features.
- Can we use DL to directly learn features from raw data?
- Yes, but we need to consider some issues.
  - To avoid loss of info, we need to sample audio data using a high sample rate 15-20KHz (44.1 Khz for music).
  - This implies lot of samples per second. Any problem?

Can We Use Raw Audio Data

- Spectrograms, log-mel, etc. are hand-crafted features.
- Can we use DL to directly learn features from raw data?
- Yes, but we need to consider some issues.
  - To avoid loss of info, we need to sample audio data using a high sample rate 15-20KHz (44.1 Khz for music).
  - This implies lot of samples per second. Any problem?
  - Using a convolutional architecture, we need huge filters or a very deep structure, why?

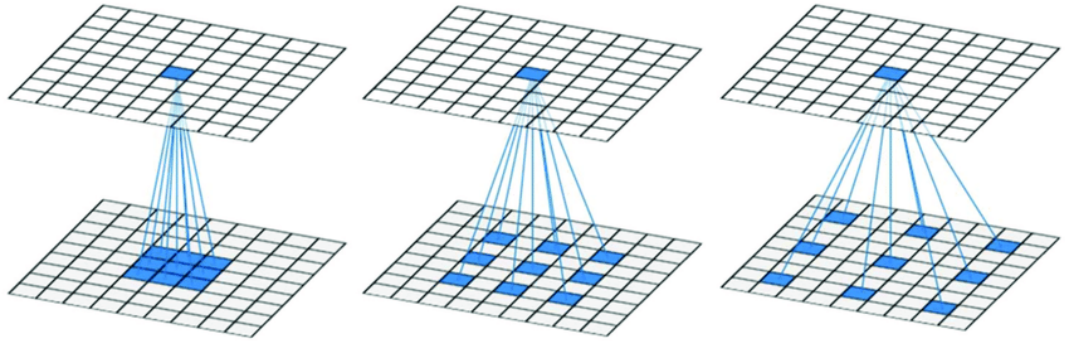
Can We Use Raw Audio Data

- Spectrograms, log-mel, etc. are hand-crafted features.
- Can we use DL to directly learn features from raw data?
- Yes, but we need to consider some issues.
  - To avoid loss of info, we need to sample audio data using a high sample rate 15-20KHz (44.1 Khz for music).
  - This implies lot of samples per second. Any problem?
  - Using a convolutional architecture, we need huge filters or a very deep structure, why?
  - We can increase the receptive field of neurons in intermediate layers using dilated convolution filters.

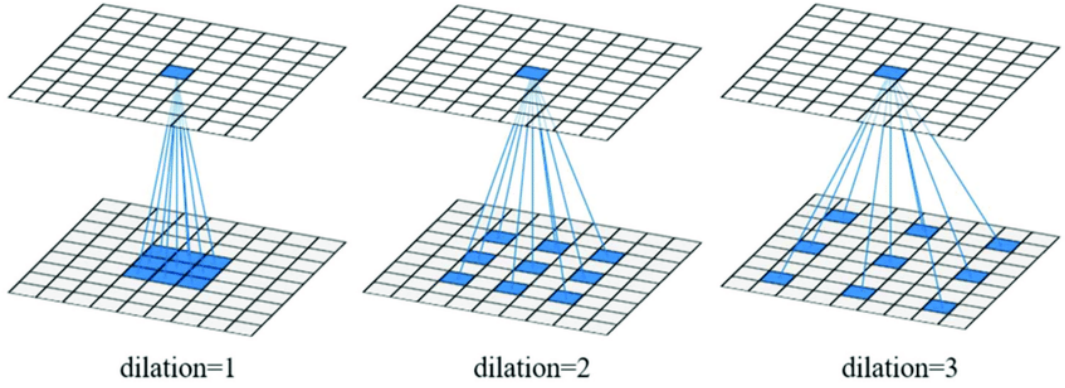
Can We Use Raw Audio Data

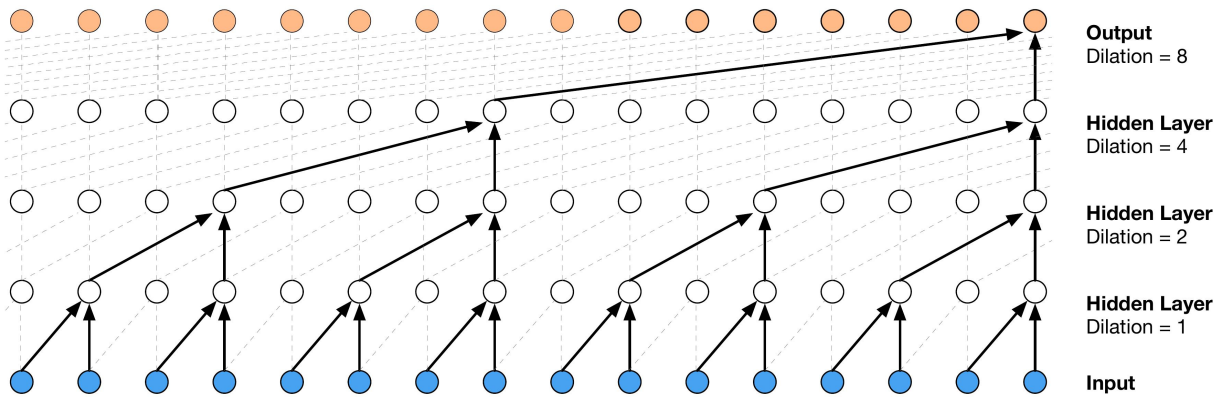
- Spectrograms, log-mel, etc. are hand-crafted features.
- Can we use DL to directly learn features from raw data?
- Yes, but we need to consider some issues.
  - To avoid loss of info, we need to sample audio data using a high sample rate 15-20KHz (44.1 Khz for music).
  - This implies lot of samples per second. Any problem?
  - Using a convolutional architecture, we need huge filters or a very deep structure, why?
  - We can increase the receptive field of neurons in intermediate layers using dilated convolution filters.
  - Dilated convolution filter ?

Dilated Convolution



Dilated Convolution





- For a CNN, reaching a sufficient receptive field size leads to a large number of parameters and a high computational complexity.
- Dilated convolutions enable CNNs to have very large receptive fields with just a few layers depth.
- In case of raw audio data, after few layers depth, neurons can cover thousands of timesteps, keeping a suitable computational efficiency.

Example 2: DL Arquiteture to Process an Audio Signal

Example 2: DL Arquiteture to Process an Audio Signal

Input:

- Raw audio 15K-20K samples per second. 1D time-series.

Example 2: DL Arquiteture to Process an Audio Signal

Input:

- Raw audio 15K-20K samples per second. 1D time-series.

CNN :

- 4 dilated convolutional layers (dilation factor depends of application). Each with: i) 128, 128, 256, 256 filters, ii) 20x1, 10x1, 10x1, 5x1 filter sizes, respectively.
- Optional max-pooling and batch normalization.

Example 2: DL Arquiteture to Process an Audio Signal

Input:

- Raw audio 15K-20K samples per second. 1D time-series.

CNN :

- 4 dilated convolutional layers (dilation factor depends of application). Each with: i) 128, 128, 256, 256 filters, ii) 20x1, 10x1, 10x1, 5x1 filter sizes, respectively.
- Optional max-pooling and batch normalization.

RNN :

- 2 LSTM layers. Cells in LSTMs with 256D.
- Need to normalize sequence length in minibatch.
- Unrolling scope depends on application.

Example 2: DL Arquiteture to Process an Audio Signal

Input:

- Raw audio 15K-20K samples per second. 1D time-series.

CNN :

- 4 dilated convolutional layers (dilation factor depends of application). Each with: i) 128, 128, 256, 256 filters, ii) 20x1, 10x1, 10x1, 5x1 filter sizes, respectively.
- Optional max-pooling and batch normalization.

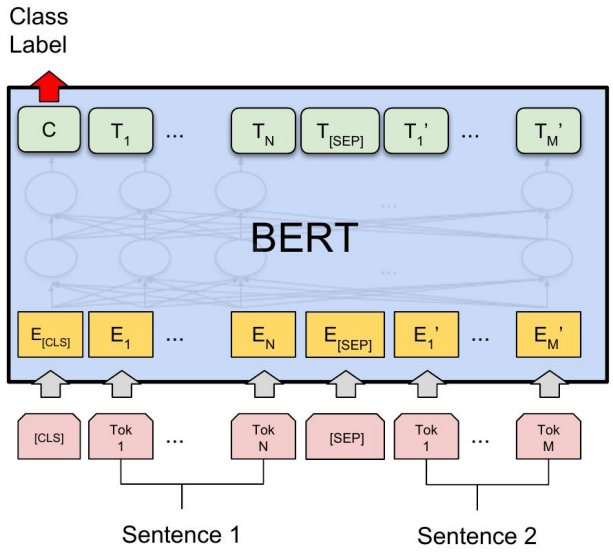
RNN :

- 2 LSTM layers. Cells in LSTMs with 256D.
- Need to normalize sequence length in minibatch.
- Unrolling scope depends on application.

MLP :

- 2 FC layers. Each FC layer has 1.024 hidden units.
- Output: softmax (or sigmoids) for class label(s).

Audio and Transformers



- There have been previous works on audio applications using a Transformer architecture. However, there are 3 relevant problems:
  1. In the context of audio, there is still a lack of highly massive audio datasets.
  2. Self-attention mechanism operates over a finite sequence of discrete entities. In the context of text, sentence segmentation is trivial, but for audio this is not the case.
  3. Transformers are not good to model long dependencies in sequences.
- As a consequence, Transformers are not currently very popular for audio applications.

Audio: Data Augmentation

Data augmentation generates additional training data by manipulating existing examples.

- In the case of audio, typical strategies include:
  - Modify pitch.
  - Add noise.
  - Perform time stretching.
- In speech recognition, a model can be pretrained on languages with more transcribed data and then adapted to a low-resource language or domain.
- Use of sound synthesis techniques is a convenient strategy. However, performance on real data might be poor if a model is trained using only synthetic data. Need for finetunning on real data.

References

- WAVENET: A generative model for raw audio, Aaron van den Oord et al., 2016.
- Convolutional, Long Short-Term Memory, fully connected Deep Neural Networks, Sainath et al., 2015.
- Deep Learning for Audio Signal Processing, Purwins et al., 2019.
- MusiCNN: Pre-trained convolutional neural networks for music audio tagging, Pons and Serra, 2019.
- Audio Set: An ontology and human-labeled dataset for audio events, Gemmeke er al., 2017.
- Transfer Learning from Speaker Verification to Multispeaker Text-To-Speech Synthesis. Jia et al., 2019.
- Scaper: a library for soundscape synthesis and augmentation. Salaman et al., 2017.